

EXP NO: 7	GENERATIVE MODELS WITH GANS: CREATING AND TRAINING A GENERATIVE ADVERSARIAL NETWORK
-----------	--

AIM:

To construct and train a **Generative Adversarial Network (GAN)** using TensorFlow/Keras on the MNIST dataset to generate new and realistic handwritten digit images.

ALGORITHM:

1. Generator:

Takes a random noise vector as input and generates synthetic handwritten digit images.

2. Discriminator:

Acts as a binary classifier that identifies whether an image is **real or fake**.

3. Training the Discriminator:

Train the discriminator using real MNIST images and fake images generated by the generator.

4. Training the Generator:

Train the generator to produce realistic images that can fool the discriminator into classifying them as real.

5. Iterative Training:

Repeat the generator and discriminator training for several epochs, allowing both networks to improve.

6. Image Generation:

Generate and display synthetic handwritten digit images at selected epochs such as **5, 10, 15, and 20**.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist
import os
```

```

(x_train, _), (_, _) = mnist.load_data()

x_train = x_train.reshape(-1, 28, 28, 1).astype("float32")
x_train = (x_train - 127.5) / 127.5

latent_dim = 100
batch_size = 256
epochs = 20
generator = keras.Sequential([
    layers.Input((latent_dim,)),
    layers.Dense(7*7*256, use_bias=False),
    layers.BatchNormalization(),
    layers.LeakyReLU(),
    layers.Reshape((7,7,256)),
    layers.Conv2DTranspose(128, 5, padding="same", use_bias=False),
    layers.BatchNormalization(),
    layers.LeakyReLU(),
    layers.Conv2DTranspose(64, 5, strides=2, padding="same", use_bias=False),
    layers.BatchNormalization(),
    layers.LeakyReLU(),
    layers.Conv2DTranspose(1, 5, strides=2, padding="same",
                          use_bias=False, activation="tanh")
])
discriminator = keras.Sequential([
    layers.Input((28,28,1)),
    layers.Conv2D(64, 5, strides=2, padding="same"),
    layers.LeakyReLU(),
    layers.Dropout(0.3),
    layers.Conv2D(128, 5, strides=2, padding="same"),
    layers.LeakyReLU(),
    layers.Dropout(0.3),
    layers.Flatten(),
    layers.Dense(1, activation="sigmoid")
])
loss_fn = keras.losses.BinaryCrossentropy()
g_opt = keras.optimizers.Adam(1e-4)
d_opt = keras.optimizers.Adam(1e-4)

@tf.function
def train_step(images):

```

```

noise = tf.random.normal((batch_size, latent_dim))

with tf.GradientTape() as gt, tf.GradientTape() as dt:
    fake = generator(noise, training=True)
    real_out = discriminator(images, training=True)
    fake_out = discriminator(fake, training=True)

    g_loss = loss_fn(tf.ones_like(fake_out), fake_out)
    d_loss = (loss_fn(tf.ones_like(real_out), real_out) +
              loss_fn(tf.zeros_like(fake_out), fake_out))

    g_grad = gt.gradient(g_loss, generator.trainable_variables)
    d_grad = dt.gradient(d_loss, discriminator.trainable_variables)

    g_opt.apply_gradients(zip(g_grad, generator.trainable_variables))
    d_opt.apply_gradients(zip(d_grad, discriminator.trainable_variables))

return g_loss, d_loss

seed = tf.random.normal((16, latent_dim))

dataset = tf.data.Dataset.from_tensor_slices(x_train)
dataset = dataset.shuffle(len(x_train)).batch(batch_size)

def generate_images(epoch):
    images = generator(seed, training=False)
    images = images * 0.5 + 0.5

    plt.figure(figsize=(4,4))
    for i in range(16):
        plt.subplot(4,4,i+1)
        plt.imshow(images[i,:,:,0], cmap="gray")
        plt.axis("off")

    plt.suptitle(f"Epoch {epoch}")
    os.makedirs("images", exist_ok=True)
    plt.savefig(f"images/epoch_{epoch}.png")
    plt.show()
    plt.close()

```

```

for epoch in range(1, epochs + 1):
    g_losses = []
    d_losses = []

    for batch in dataset:
        g, d = train_step(batch)
        g_losses.append(g.numpy())
        d_losses.append(d.numpy())
    print(f'Epoch {epoch}/20 - G Loss: {np.mean(g_losses):.4f} - D Loss:
{np.mean(d_losses):.4f}')

    if epoch in [5, 10, 15, 20]:
        generate_images(epoch)
print("Training complete!")

```

OUTPUT:

```

--- Part 1: Loading and Preprocessing the MNIST Dataset ---
Normalized training data shape: (60000, 28, 28, 1)
Example of a normalized pixel value: -1.0

--- Part 2: Building the GAN Components ---

--- Generator Model Summary ---

--- Discriminator Model Summary ---

--- Beginning GAN Training ---
Epoch 1/20 - Generator Loss: 0.8775, Discriminator Loss: 0.8910
Epoch 2/20 - Generator Loss: 0.9409, Discriminator Loss: 1.1884
Epoch 3/20 - Generator Loss: 0.9654, Discriminator Loss: 1.1334
Epoch 4/20 - Generator Loss: 0.8210, Discriminator Loss: 1.2918
Epoch 5/20 - Generator Loss: 0.9268, Discriminator Loss: 1.2375

```

Epoch 5



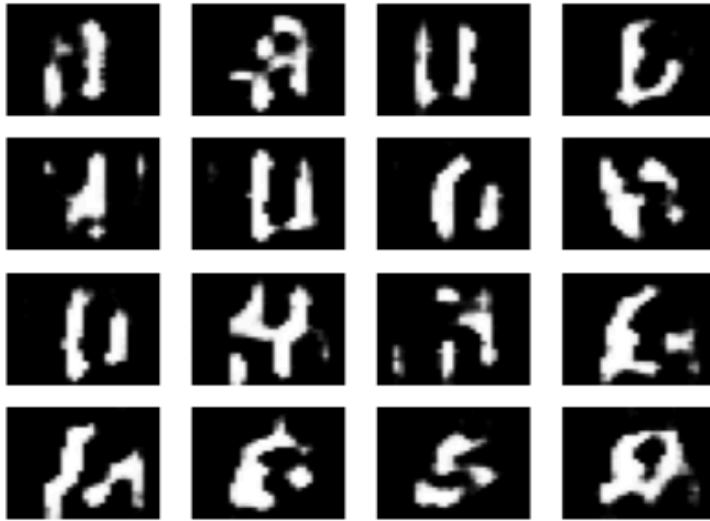
Epoch 6/20 - Generator Loss: 0.8511, Discriminator Loss: 1.2915
Epoch 7/20 - Generator Loss: 0.9092, Discriminator Loss: 1.2378
Epoch 8/20 - Generator Loss: 0.8859, Discriminator Loss: 1.2685
Epoch 9/20 - Generator Loss: 0.8709, Discriminator Loss: 1.3010
Epoch 10/20 - Generator Loss: 0.9137, Discriminator Loss: 1.2243

Epoch 10



```
Epoch 11/20 - Generator Loss: 0.9682, Discriminator Loss: 1.1675
Epoch 12/20 - Generator Loss: 1.0014, Discriminator Loss: 1.1624
Epoch 13/20 - Generator Loss: 1.0475, Discriminator Loss: 1.1707
Epoch 14/20 - Generator Loss: 1.0859, Discriminator Loss: 1.1226
Epoch 15/20 - Generator Loss: 1.2063, Discriminator Loss: 1.0021
```

Epoch 15



```
Epoch 16/20 - Generator Loss: 1.1530, Discriminator Loss: 1.0588
Epoch 17/20 - Generator Loss: 1.1024, Discriminator Loss: 1.1346
Epoch 18/20 - Generator Loss: 1.1340, Discriminator Loss: 1.0879
Epoch 19/20 - Generator Loss: 1.1815, Discriminator Loss: 1.0992
Epoch 20/20 - Generator Loss: 1.1595, Discriminator Loss: 1.0578
```

Epoch 20



--- Training complete ---

RESULT:

The GAN was successfully trained and generated **realistic synthetic handwritten digit images**.